Paid Media Full Funnel

Author: Michael Dey
Created: October 14, 2025 at 02:35 PM
Last Updated: February 3, 2026 at 05:05 PM
Updated By: Elle Su (Deactivated)
Version: 25
Source: https://redventures.atlassian.net/wiki/spaces/DEBR/pages/1019216201118/Paid+Media+Full+Funnel
Exported: March 31, 2026 at 04:55 PM

Table of Contents

- Overview
- Purpose
- s (BI)
- Columns
- Sample Queries
- Differences Between Old & New Tables/Improvements Made

Overview

The following documentation highlights a high-level understanding and tour of the paid media full funnel project as of 12/09/2025 and how it differs from the existing methodology.

Purpose

The goal of these tables is to make it easier to pull paid media data and answer questions like “how much money did paid media make yesterday?”

s (BI)

- Currently, the tables are pulling from the old conversion tables before 8/1/2025, and is backfilled to 1/1/2024.
- To backfill the table further, the date filters in the “bf_” subqueries of thematerialized viewshould be adjusted (low lift)2024 was used as a cutoff for performance reasons
- The vertical column was built off of logic based on the account_name and in some cases campaign_name logic. If there are numbers not adding up for a specific vertical, check vertical = 'other' to see if any campaigns were misclassified.
- The cards bottom of funnel logic is filtered on utm_pid (currently including brcctaboola,sembrcc, and brccsocial.) If new platforms are added or pids change, this will need to be adjusted for offer clicks and approvals to be in the table.

Columns

paid_media_ff_campaign_daily (Gold Layer)

Column Name	Description
platform	Ad platform (Google, Meta, Microsoft, Taboola, etc.)

channel	Type of paid media (Paid Search, Paid Native, Paid Social, Paid Display)
date	Date of spend and revenue
campaign_id	Campaign ID from the UI
vertical	Categorizes campaigns by vertical; used for categorizing spend by vertical for accruals
account_id	Ads account ID
account_name	Ads account name
search	True if the campaign is a search campaign, false otherwise; distinct from channel and correctly categorizes Pmax campaigns as non-search
cost	Spend from the UI
impressions	Number of impressions
reach	Distinct users who saw ads (paid other)
clicks	Number of clicks
engagements	Number of interactions with ads (paid other)
conversions	Number of conversions from the UI
conversion_value	Conversion value from the UI (could be inflated)
video_starts	Number of video starts (paid other)
video_views_25	Number of video ads where >= 25% has been viewed (paid other)
video_views_50	Number of video ads where >= 50% has been viewed (paid other)
video_views_75	Number of video ads where >= 75% has been viewed (paid other)
video_completions	Number of video ads where 100% has been viewed (paid other)
video_views_3_seconds	Number of video ads where >=3 seconds has been viewed (paid other)
total_actual_revenue	Sum of revenue pulled from conversions tables; source of truth

Sample Queries

Full Funnel

```
select  vertical,  case    when search = true then 'Paid Search' else 'Paid Other'  end as channel,  sum(impressions) as impressions,  sum(clicks) as clicks,  sum(cost) as cost,  sum(total_actual_revenue) as revenue,  case when vertical = 'investing' then sum(investing_lead_sold)  else sum(total_cpl_leads) end as leads,  sum(total_cpc_clicks) as rpc_clicksfrom  bankrate_prod.br_rpt.paid_media_ff_campaign_dailywhere  date between  date_trunc('month', add_months(current_date(), -1))  and    date_trunc('month', current_date()) - interval 1 day  and account_name <> 'Safeco Insurance'group by  1,  2
```

Accruals

```
select  vertical,  platform,  sum(cost) as costfrom  bankrate_prod.br_rpt.paid_media_ff_campaign_dailywhere  date between    date_trunc('month', add_months(current_date(), -1))  and    date_trunc('month', current_date()) - interval 1
```

```
day  and account_name <> 'Safeco Insurance'  and vertical <> 'ndr'  and vertical <> 'other'
and vertical <> 'main'group by  1,  2
```

Accruals - Pivoted

```
select *from (  select    vertical,    platform,    cost  from
bankrate_prod.br_rpt.paid_media_ff_campaign_daily  where    date between
date_trunc('month', add_months(current_date(), -1))        and date_trunc('month',
current_date()) - interval 1 day    and account_name <> 'Safeco Insurance'    and vertical
not in ('ndr', 'other', 'main'))pivot (  sum(cost)  for platform in ('Google', 'Meta',
'Microsoft', 'Taboola'))order by vertical;
```

Accruals - Pure Brand

```
select  platform,  sum(deposits_revenue / total_actual_revenue * cost)        as deposits,
sum(cards_revenue / total_actual_revenue * cost)           as cards,
sum(homeequity_total_revenue / total_actual_revenue * cost) as home_equity,
sum(investing_revenue / total_actual_revenue * cost)       as investing,
sum(insurance_revenue / total_actual_revenue * cost)  as insurance,
sum(loans_total_revenue / total_actual_revenue * cost)  as loans,
sum(mortgage_total_revenue / total_actual_revenue * cost)  as mortgages,  sum(cost) as
totalfrom bankrate_prod.br_rpt.paid_media_ff_campaign_dailywhere vertical = 'main'  and
date between date_trunc('month', add_months(current_date(), -1))              and
date_trunc('month', current_date()) - interval 1 daygroup by 1;
```

How to Use the New Tables

There are two primary tables relevant to paid media full funnel analysis: paid_media_conversions_enriched_all_vertical and paid_media_daily. These two tables comprise what can be referred to as the “silver layer” in[medallion architecture](). Between these two tables, any reporting table (“gold layer”) can be created through basic queries and no additional ETL logic. The primary key on each table is as follows:

- paid_media_conversions_enriched_all_verticals: conversions_core_pk(a derived key based off unique fields for conversions events. These fields are different for each vertical)
- paid_media_daily: paid_media_daily_pk defined as:

```
md5(concat(         coalesce(ad_id, ''),  coalesce(ad_name, ''),  coalesce(ad_group_id,
-1)::string,  coalesce(asset_group_id, -1)::string,  coalesce(campaign_id, ''),
coalesce(keyword_id, ''),  coalesce(device, ''),  coalesce(date, '1900-01-01')::string))
```

Utm fields are sparsely populated throughout the conversions table as these data are not necessarily gathered or available for all sessions. To give a rough idea of this, see the screenshot below:

vertical	utm_adid_pct_null	utm_adgid_pct_null	utm_campaignid_pct_null	utm_keyword_pct_null	utm_device_pct_null
credit cards	96.3	95.6	93.4	95.9	93.5
deposits	65.2	63.6	58.2	62.9	64.6
homeequity	88.9	83.2	82.5	87.6	86.4
insurance	99.7	99.7	99.6	99.8	99.7
investing	79.4	78.8	78.4	78.9	78.9
loans	96.7	96.0	95.0	95.6	96.6
mortgage	91.5	87.9	82.1	83.5	88.6
realestate	87.3	78.0	56.6	57.7	79.2

image-20251024-175738.png

In general, the granularity of the conversions table is higher than that of the paid_media table. For any given unique combination in the paid_media table, there will be numerous purchase ids associated with it in the conversions table.

```
with t as (  select     *,    md5(concat(             coalesce(utm_adid, ''),      '', --
no ad_name      coalesce(utm_adgid, -1)::string,      '-1', -- no asset_group_id
coalesce(utm_campaignid, ''),      coalesce(utm_keywordid, ''),      coalesce(CASE
WHEN utm_device in ('m', 'Smartphone') THEN 'MOBILE'        WHEN utm_device in ('t',
'Tablet') THEN 'TABLET'        WHEN utm_device in ('c', 'Desktop') THEN 'DESKTOP'
ELSE 'blank'      END, ''),      coalesce(date, '1900-01-01')::string    )) as
paid_media_daily_pk  from conversions_enriched_all_verticals)select utm_adid, utm_adgid,
```

```
utm_campaignid, utm_keywordid, utm_device, date, *from twhere paid_media_daily_pk in (
select paid_media_daily_pk  from t  where paid_media_daily_pk in (    select
paid_media_daily_pk    from paid_media_daily  )  group by 1 having count(*) > 1  limit 1)
```

Thus, the only way to achieve a 1-to-1 match between the two tables (and not create duplicates or information loss) is to aggregate each before joining. Any combination of the following fields can be aggregated first then joined:

paid_media_daily	conversions_enriched_all_verticals
ad_id	utm_adid
ad_group_id	utm_adgid
campaign_id	utm_campaignid
keyword_id	utm_keywordid
device	CASE WHEN utm_device in ('m', 'Smartphone') THEN 'MOBILE' WHEN utm_device in ('t', 'Tablet') THEN 'TABLET' WHEN utm_device in ('c', 'Desktop') THEN 'DESKTOP' ELSE 'blank' END
date	date

Below is an example of what could happen if the tables are not pre-aggregated to the same fields. This example uses campaign_id and date for paid_media_daily and campaign_id, date, and vertical for conversions:

Table +

Duplicate costs | unique revenue

	campaign_id	date	1.2 cost	vertical	1.2 actual_revenue
1	12683436456	2025-08-14	21.56	loans	38
2	12683436456	2025-08-14	21.56	homeequity	0
3	12683436456	2025-08-14	21.56	realestate	120
4	12683436456	2025-08-14	21.56	mortgage	17589.22

image-20251022-150056.png

Since the paid media table cannot have vertical information, it's not possible to accurately attribute fractions of the total cost to each revenue outcome. Instead, a pivot method should be used to accurately reflect and preserve vertical information when joining:

Table +

	campaign_id	date	1.2 cost	1.2 loans_total_revenue	1.2 homeequity_total_revenue	1.2 realestate_total_revenue	1.2 mortgage_total_revenue
1	12683436456	2025-08-14	21.56	38	0	120	17589.219999999998

image-20251022-150552.png

The above is achieved through case conditions as seen below but using the[pivot clause](#)is also acceptable:

```
with pm as (  select    campaign_id,    date,    sum(cost) as cost  from
br_rpt.paid_media_daily  group by 1,2),con as (  select    utm_campaignid,    date,
sum(case when vertical = 'loans' then actual_revenue else 0 end) as loans_total_revenue,
sum(case when vertical = 'homeequity' then actual_revenue else 0 end) as
homeequity_total_revenue,    sum(case when vertical = 'realestate' then actual_revenue else
0 end) as realestate_total_revenue,    sum(case when vertical = 'mortgage' then
actual_revenue else 0 end) as mortgage_total_revenue  from
br_rpt.conversions_enriched_all_verticals  group by 1,2)select  ad.*,  pm.* except
(utm_campaignid, pm.date)from pmleft join con  on pm.campaign_id = con.utm_campaignid  and
pm.date = con.date
```

The above methodology is the accepted approach needed to properly join and capture data with minimal loss of information and is used in br_rpt.paid_media_ff_campaign_daily. This materialized view establishes a use-case of the silver layer and how to convert the architected layer into reporting-friendly views in the gold layer.

Differences Between New and Old Tables

Structural and functional changes

A good comparison of the old tables vs new tables used to capture paid media full funnel can be seenhere.

Deprecated fields and new fields

A comprehensive look at the new field list can be foundhere.

Improvements Made

conversions_enriched_paid_media

Unlike previous tables (such as clicksanalytics_v2), conversions_enriched_paid_media table does an exhaustive search of any and all paid-search / UTM parameters that could possibly be tied to an event. The search criteria for any given field is as follows

- Check any columns within the conversions_core table that may have the value. For example, for utm_medium, the following columns are checked in order:metadata_wealth.meta_clickData_referrerUrl
- metadata_homelending.meta_marketingData
- chargeable_wc_page_url
- ps_utms.utmMedium
- Check any columns within sessions_fact (joined to conversions_core on session_id if present. Session_id being derived by coalescing 5 columns within conversions_corefirstpageurl
- firstpagereferrer
- lastpageurl
- lastpagereferrer
- Check any columns within pageview_fact (joined on instance_id). Instance_id derived by coalescing 5 columns in conversions_core:pageurl
- pagereferrer

One other change to be mentioned is the default timezone columns in conversions_core is UTC and that for clicks is EST. conversions_enriched_paid_media was also switched to EST for easy comparison against clicksanalytics and more consistent results. What this means:

- The "pageurl" or "session_referrer_url" column are not guaranteed to match the url in which the utm / ps value was found (it returns the first non-null value from the above order of precedence which could differ for each)
- This should provide >= the number of available ps / utm params from any of the one tables alone meaning if you use a filter such as utm_medium on the conversions_enriched_paid_media table, you're more likely to return a match than any one of the origin tables by itself
- Bullets 1 and 2 make it difficult to compare, for example, revenue by campaign since you can't just use clicksanalytics columns or conversions_core columns in isolation but would

need to also join to sessions_fact and pageview_fact and perform all the same coalesce operations.

paid_media_daily

For the paid_media_daily table, a raw (but deduplicated) consolidation of all paid media sources is concatenated together into one comprehensive view. This includes both ad_group and asset_group data as to not omit any needed campaign data.

New Capabilities

The primary idea of these two tables is to be able to answer any full-funnel question without needlessly complex logic to do so. These two tables consolidate nearly all potential full-funnel fields relevant to paid media.